Paired diagnostic report for RefCOCOg VLM grounding

Abstract

This report describes a paired diagnostic of four Qwen3-VL-8B-Instruct systems for 2D RefCOCOg referring-expression grounding. The systems were evaluated on the same 500 training examples and the same 200 validation examples using corrected version 2 manifests. The validation set is a small diagnostic subset, so these results are useful for this experiment but are not an official benchmark or leaderboard result. SFT produced the largest validation gain over the base model. Direct GRPO produced a smaller gain. GRPO after SFT was close to SFT on validation and had the highest mean IoU on the 500-example training set.

1 Purpose and scope

The repository studies referring-expression grounding with a vision-language model. Given an image and a phrase describing one object, the model must return one bounding box. The code uses Qwen3-VL-8B-Instruct and supports a base evaluation, LoRA supervised fine-tuning (SFT), direct GRPO, and GRPO initialized from an SFT adapter.

The experiment in this report answers a narrow question: how do these three training variants compare with the base model when every system is evaluated on the same examples and annotations? It does not estimate a leaderboard score, generalization across RefCOCOg splits, or the effect of multiple random seeds.

2 Data and paired evaluation

The diagnostic used the following corrected manifests:

- `refcocog_train500_v2.jsonl`, with 500 rows;
- `refcocog_validation200_v2.jsonl`, with 200 rows.

The two manifests have zero overlapping reference IDs. Within each split, all four systems were evaluated on the same rows. The diagnostic script pairs rows by `ref_id` when available and checks the phrase and ground-truth pixel box before calculating a comparison. The paired comparisons summarized here were all comparable.

Earlier comparisons used mismatched manifests. Those comparisons were superseded by this version 2 paired diagnostic and are not used in the results below.

3 Systems and training setup

All systems use Qwen3-VL-8B-Instruct. The four configurations are:

Base. The base model without task training.

SFT. LoRA supervised fine-tuning on the converted RefCOCOg examples.

Direct GRPO. GRPO started from the base model.

SFT then GRPO. GRPO started from the SFT adapter and used that adapter as the reference adapter.

The checked-in Slurm launcher uses bfloat16, LoRA rank 8, LoRA alpha 32, `all-linear` target modules, maximum sequence length 2048, and one example per device batch. SFT runs for one epoch with learning rate `1e-4`, gradient accumulation 8, and the SFT JSONL data. The GRPO runs use learning rate `5e-6`, four generations, `beta=0.04`, maximum completion length 64, gradient accumulation 4, and the GRPO JSONL data.

The prompt asks for one box in the form `[[x1, y1, x2, y2]]`, with coordinates on a 0 to 1000 scale relative to the image width and height. The data converter stores the target box in cached-image pixel coordinates for the reward function. The evaluator converts valid normalized predictions to the same pixel space before computing IoU.

The GRPO reward is pixel-space IoU for a valid box and `-0.1` for an invalid box. The evaluator records invalid or failed predictions as `parse_ok=false` and assigns them IoU 0.0.

4 Metrics

For each prediction file, the report uses:

Mean IoU. The mean intersection over union in cached-image pixel space.

Acc@0.5. The fraction of examples with IoU at least 0.5.

Acc@0.75. The fraction of examples with IoU at least 0.75.

Parse failures. The number of examples without a valid parsed box.

The diagnostic also reports the mean predicted-area to ground-truth-area ratio and mean center distance for parsed rows. The paired tables below focus on the requested accuracy metrics and mean IoU. A paired mean-IoU delta is the variant IoU minus the base IoU on each identical row, averaged over the split. The improved, degraded, and unchanged counts classify those per-row deltas.

5 Results

5.1 Validation results

Table 1 reports the 200-example validation diagnostic.

Table 1: Validation metrics on 200 paired examples.

System	Mean IoU	Acc@0.5	Acc@0.75	Parse failures
Base	0.8023357	0.845	0.780	1
SFT	0.8160114	0.860	0.800	0
Direct GRPO	0.8042084	0.850	0.780	1
SFT then GRPO	0.8158485	0.860	0.795	0

Relative to the base validation run, the paired mean-IoU deltas were:

Table 2: Paired validation mean-IoU changes against the base run.

Variant	Delta	Improved	Degraded	Unchanged	Parse failures
SFT	+0.01367575	101	97	2	0
Direct GRPO	+0.00187275	44	44	112	1
SFT then GRPO	+0.01351283	99	99	2	0

SFT has the highest validation mean IoU and Acc@0.75 among the trained variants. SFT then GRPO has the same Acc@0.5 as SFT and a slightly lower mean IoU and Acc@0.75 on this split. Direct GRPO is close to the base model: its mean IoU is higher by 0.00187275, its Acc@0.5 is higher by 0.005, and its Acc@0.75 is unchanged. The base and direct GRPO runs each have one parse failure. SFT and SFT then GRPO have none.

5.2 Training-split results

Table 3 reports the same systems on the 500-example training diagnostic.

Table 3: Training-split metrics on 500 paired examples.

System	Mean IoU	Acc@0.5	Acc@0.75	Parse failures
Base	0.7976700	0.858	0.744	0
SFT	0.8251656	0.890	0.782	0
Direct GRPO	0.8019457	0.864	0.748	0
SFT then GRPO	0.8291169	0.890	0.794	0

The paired training-split changes were:

Table 4: Paired training-split mean-IoU changes against the base run.

Variant	Delta	Improved	Degraded	Unchanged
SFT	+0.02749562	258	237	5
Direct GRPO	+0.00427569	105	96	299
SFT then GRPO	+0.03144688	278	216	6

SFT then GRPO has the highest training mean IoU and Acc@0.75. SFT and SFT then GRPO both reach Acc@0.5 of 0.890. Direct GRPO again has a smaller change from base than SFT, with a paired mean-IoU delta of +0.00427569.

5.3 Training run records

The recorded training runs completed the following steps:

The SFT record contains token accuracy, while the GRPO records contain a final logged reward. The objectives, logging values, and training procedures differ, so the train losses and final values should not be ranked across methods.

6 Analysis

The paired results support three limited observations.

Table 5: Training run records. These values are not comparable across methods.

Run	Steps	Runtime (s)	Train loss	Final logged value
SFT	63/63	130.4228	0.89258396	token accuracy 0.69059829
Direct GRPO	500/500	2134.9276	0.00159036	reward 0.68845888
SFT then GRPO	500/500	2531.4886	0.00034432	reward 0.69928728

First, SFT gives the clearest improvement over the base model in both measured splits. Its mean-IoU delta is +0.01367575 on validation and +0.02749562 on the training diagnostic. The validation Acc@0.5 rises from 0.845 to 0.860, and Acc@0.75 rises from 0.780 to 0.800.

Second, direct GRPO gives a smaller change in this run. Its validation paired delta is +0.00187275, with 44 improved rows, 44 degraded rows, and 112 unchanged rows. The training paired delta is +0.00427569, with 105 improved, 96 degraded, and 299 unchanged rows. These results do not show a large direct GRPO improvement over the base model on the evaluated examples.

Third, SFT then GRPO remains close to SFT on validation and exceeds it on the training diagnostic. On validation, SFT then GRPO has mean IoU 0.8158485, compared with 0.8160114 for SFT. On the training diagnostic, it has mean IoU 0.8291169, compared with 0.8251656 for SFT. This difference is descriptive; the experiment does not include repeated runs or a larger held-out benchmark that would establish whether the difference is reliable.

The parse-failure counts are low. Validation has one failure for the base and direct GRPO runs and none for SFT or SFT then GRPO. All four training-split runs have zero parse failures. Because invalid parses receive IoU 0, parse reliability is part of the reported mean, not a separate post-processing adjustment.

7 Limitations

The validation diagnostic contains only 200 examples. It is not a definitive benchmark, and the results should not be presented as an official RefCOCOg leaderboard result. The training split is used here as a diagnostic comparison, not as evidence of held-out generalization.

The experiment reports one run for each configuration. It does not measure variation across random seeds, compare multiple hyperparameter settings, or provide statistical uncertainty. The paired comparison reduces variation from example composition, but it does not remove these other sources of uncertainty.

The report includes mean IoU, threshold accuracies, parse failures, and paired win/loss counts. The diagnostic JSON also contains area-ratio and center-distance statistics. These box-shape measures are useful for follow-up analysis, but they do not replace a larger evaluation.

8 Artifacts and reproduction

The completed diagnostic artifacts remain on the HPC filesystem and are not tracked in this repository:

```
/scratch/ambpdc/soham/vlm/outputs/diagnostic/
base_train.jsonl
```

```
sft_train.jsonl
grpo_train_h100.jsonl
sft_grpo_train_h100.jsonl
base_val.jsonl
sft_val.jsonl
grpo_val.jsonl
sft_grpo_val.jsonl
full_report.json
```

The report can be reproduced from the repository with the following workflow. The commands assume the HPC layout used by the Slurm launcher.

1. Install the lightweight dependencies from `requirements.txt`.
2. Prepare the RefCOCOg image caches and the version 2 train and validation manifests with `scripts/prepare_refcocog_images.py`.
3. Convert each manifest with `scripts/prepare_swift_data.py`.
4. Run the three training modes from the repository checkout: `sft`, `grpo_base`, and `grpo_after_sft`.
5. Evaluate the base model and each adapter with `scripts/evaluate_grounding.py`, using the same manifest within each split.
6. Run `scripts/analyze_grounding_diagnostic.py` with the eight prediction files and write `full_report.json`.

The paired diagnostic command is:

```
D=/scratch/ambpdc/soham/vlm/outputs/diagnostic
python3 scripts/analyze_grounding_diagnostic.py \
  --prediction base/train=$D/base_train.jsonl \
  --prediction sft/train=$D/sft_train.jsonl \
  --prediction grpo/train=$D/grpo_train_h100.jsonl \
  --prediction sft_grpo/train=$D/sft_grpo_train_h100.jsonl \
  --prediction base/val=$D/base_val.jsonl \
  --prediction sft/val=$D/sft_val.jsonl \
  --prediction grpo/val=$D/grpo_val.jsonl \
  --prediction sft_grpo/val=$D/sft_grpo_val.jsonl \
  --output $D/full_report.json
```

Model weights, images, manifests, adapters, logs, and prediction files are external artifacts. Their absence from Git is intentional.

9 Conclusion

On this paired diagnostic, SFT produced the largest validation improvement over the base model. SFT then GRPO was close to SFT, while direct GRPO changed little.